

P3178R1

Retrieval of Exception Information

New Proposal, 2024-05-30

This version:

<http://virjacode.com/papers/p3178r1.htm>

Latest version:

<http://virjacode.com/papers/p3178.htm>

Author:

Thomas PK Healy <healytpk@vir7ja7code7.com> *(Remove all sevens from email address)*

Audience:

SG18 Library Evolution Working Group (LEWG)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Add two new functions to the standard library to facilitate the retrieval of further information about the currently handled exception, or from an `exception_ptr`.

Declared in header `<exception>`

```
type_info const &exception_typeid() noexcept; (1) (
```

```
type_info const &exception_typeid(exception_ptr const &p) noexcept; (2) (
```

```
void *exception_object() noexcept; (3) (
```

```
void *exception_object(exception_ptr const &p) noexcept; (4) (
```

- 1) Retrieve the `std::type_info` for the currently handled exception
- 2) Retrieve the `std::type_info` for the exception referred by `p`
- 3) Retrieve the address of the exception object for the currently handled exception

4) Retrieve the address of the exception object for the exception referred by **p**

Parameters

p - null or non-null `std::exception_ptr`

Return value

- 1,2) The `std::type_info` of the exception, otherwise `typeid(void)` if unavailable.
- 3,4) The address of the exception object, otherwise `nullptr` if unavailable.

Table of Contents

1 Introduction

2 Motivation

2.1 `catch(...)`

2.2 Runtime plugins

2.3 As a stepping stone

3 Impact on the standard

4 Impact on existing code

5 Design considerations

5.1 Microsoft SEH

5.2 Forced unwinding

6 Possible implementations

6.1 Apple macOS, Linux, FreeBSD

6.2 Microsoft Windows

7 Proposed wording

8 Alternatives

8.1 `std::exception_ptr::type()`

9 Acknowledgements

10 Revision history

References

Normative References

Informative References

§ 1. Introduction

Inside a `catch(...)` block, we should have access to the `type_info` of what was thrown. Currently there is no portable way to get the `type_info` from an `exception_ptr`, and so if an object is thrown of intrinsic type or a non-polymorphic class type, and caught in a `catch(...)` block, the type of the exception object is unknown. It is also impossible to get the address of the exception object itself.

§ 2. Motivation

§ 2.1. `catch(...)`

A C++ program can link with a shared library which has incomplete or inaccurate documentation for the exceptions it throws, or the program may link at runtime using `LoadLibrary` or `dlopen` to link with a library which it knows very little about. Code in the main program may handle an exception thrown from within the shared library as follows:

```
#include <exception> // exception, exception_ptr
#include <new>        // bad_alloc
#include <typeinfo>   // typeid, type_info

extern void SomeLibraryFunction(void) noexcept(false);

int main(void)
{
    try
    {
        SomeLibraryFunction();
    }
    catch(std::bad_alloc const&)
    {
        // Attempt safe shutdown when memory is depleted
    }
}
```

```

    }
    catch(std::exception const &e)
    {
        std::type_info const &ti = typeid(e);
        // As 'std::exception' has a virtual destructor, it is
        // a polymorphic type, and so 'typeid' will give us the
        // RTTI of the derived class.
    }
    catch(...)
    {
        std::exception_ptr const p = std::current_exception();
        // We can get a pointer to the exception object, but
        // we have no idea of the type of what was thrown
    }
}

```

This proposal will allow us to access the `type_info` of the exception object inside a `catch(...)` block, as follows:

```

catch(...)
{
    std::type_info const &ti = std::exception_typeid();
}

```

And also to get the `type_info` from any `exception_ptr`, as follows:

```

void Func(std::exception_ptr const &p)
{
    std::type_info const &ti = std::exception_typeid(p);
    std::puts( ti.what() );
}

int main(void)
{
    std::exception_ptr p;

    try
    {
        throw std::runtime_error("resources depleted");
    }
    catch(...)

```

```

    {
        p = std::current_exception();
    }

    Func(p);
}

```

Once we have the `type_info`, we can perform checks on the name:

```

catch(...)
{
    std::type_info const &ti = std::exception_typeid();
    if ( std::strstr( ti.name(), "FieryShadow" ) ) return EXIT_FAILURE;
}

```

The name of the type will of course be mangled and therefore be different on different compilers, but if a class is named **'FieryShadow'**, then its mangled name will contain **'FieryShadow'**, hence the function `std::strstr` will work here. However, until a name mangling library is added to the standard library, code to distinguish specifically between type names (e.g. where one class is inside an anonymous namespace) will be platform-specific and non-portable.

§ 2.2. Runtime plugins

A massive program such as 3D modelling software for a desktop computer may allow the installation of 3rd party plugins. These plugins come in the form of a dynamic shared library (e.g. **'dll'** on *Microsoft Windows*, or **'so'** on *Apple macOS*) which is loaded at runtime using `LoadLibrary` or `dlopen`, and the plugin must export a list of functions expected by the main program, e.g. `Plugin_GetName()` and `Plugin_GetCategory()`. One of these exported functions could be `Plugin_GetExceptions()` as follows:

```

std::type_info const *const *Plugin_GetExceptions();

```

`Plugin_GetExceptions()` returns a pointer to a null-terminated array of pointers to the `type_info`'s of bespoke exceptions thrown from the plugin. When the main program first loads the plugin, the main program populates a global array of these exception types, and the main program consults this array when an exception is caught:

from the following code:

```
struct Dummy {
    char buf[32];
    Dummy(void) : buf("Frogs123") {}
};

try
{
    throw Dummy();
}
catch(...)
{
    using std::exception_object;
    using std::sizeof_type_info;

    std::type_info const &ti = std::exception_typeid();
    printf("Type of exception object: %s\n" , ti.name());

    if ( (0u==sizeof_type_info(ti)) || (-1==sizeof_type_info(ti)) ) return;
    printf("Size of exception object: %zu\n", sizeof_type_info(ti));

    char unsigned const *const p = static_cast<char unsigned*>(exception_o
    printf("  Hex: ");
    for ( size_t i = 0u; i < sizeof_type_info(ti); ++i )
    {
        printf("%02x", static_cast(p[i]) );
    }
    printf("\nASCII: ");
    for ( size_t i = 0u; i < sizeof_type_info(ti); ++i )
    {
        char c = p[i];
        if ( !isprint(c) || isspace(c) ) c = ' ';
        putchar(c);
    }
    putchar('\n');
}
```

This would greatly aid in logging exceptions in both *Debug* and *Release* builds, even where C++ code is linked with another language such as Java or C#.

§ 3. Impact on the standard

This proposal is a pure library extension. It does not require changes to the standard components. Four short paragraphs are to be appended to Section 17.9.7 [\[support.exception.propagation\]](#). The text addition is 139 words, and the addition has no effect on any other part of the standard.

§ 4. Impact on existing code

No existing code becomes ill-formed. The behaviour of all existing code is unaffected by this addition to the standard library.

§ 5. Design considerations

§ 5.1. Microsoft SEH

The Microsoft C++ compiler has two kinds of exception:

- C++ exceptions
- SEH exceptions

SEH exceptions are not caught by a C++ `catch` block -- not even by a `catch(...)` block. However if you compile with the `/EHa` switch, an SEH exception will be caught by a `catch(...)` block. In this case, `current_exception()` returns a valid `exception_ptr`, however there is no `type_info` available for an SEH exception. In any circumstance where `type_info` is unavailable, the recommendation is that `exception_typeid()` and `exception_typeid(p)` shall return `typeid(void)`, or alternatively the `type_info` of a special type such as `typeid(_s_se_exception)`.

`current_exception()` returns a valid pointer for an SEH exception, and the exception object is an `unsigned int`. The recommendation therefore is that `std::exception_object()` shall return the address of the `unsigned int`.

There is a workaround to ensure that SEH exceptions have `type_info`, and that is to use the function `_set_se_translator` to convert *SEH exceptions* to *C++ exceptions* as follows:

```
// Compile with Microsoft C++ compiler with option /EHa
#include <cstdio>      // std::puts, ::sprintf_s
#include <exception>   // exception
```



```

#include <eh.h>          // _set_se_translator, _EXCEPTION_POINTERS

class SEH_Exception : public std::exception {
private:
    unsigned const var_number;
    mutable char var_what[14u + 1u]; // +1 for null terminator
public:
    SEH_Exception(unsigned const arg) noexcept : var_number(arg) {}
    char const *what(void) const noexcept override
    {
        static_assert( 0xFFFFFFFF == (unsigned)-1, "unsigned int must be 32 bits");
        ::sprintf_s(this->var_what, "SEH 0x%08x", this->var_number);
        return this->var_what;
    }
    unsigned number(void) const noexcept { return var_number; }
};

void DivideByZero(void)    // Throws an SEH exception
{
    int volatile x, y = 0;
    x = 5 / y;              // division by zero throws 0xc0000094
}

void translator(unsigned const u, ::_EXCEPTION_POINTERS*)
{
    throw SEH_Exception(u);
}

int main(void)
{
    ::_set_se_translator( translator );

    try
    {
        DivideByZero();
    }
    catch(std::exception const &e)
    {
        std::puts( e.what() );
    }
}

```

§ 5.2. Forced unwinding

The throwing of an exception results in unwinding of the stack, and so normally stack unwinding is accompanied by the presence of an exception object. There are circumstances however in which stack unwinding can take place without an accompanying exception object, and one such circumstance is a **forced unwinding**. An example of a forced unwinding is where the execution of a thread is abruptly stopped by the invocation of `pthread_cancel`, as follows:

```
#include <stdio>           // puts
#include <exception>        // exception_ptr
#include <pthread.h>        // pthread_create
#include <unistd.h>         // sleep

void *Thread_Entry_Point(void*)
{
    try
    {
        for (; /* ever */ ;)
        {
            std::puts("*");
            ::sleep(1u);
        }
    }
    catch(...)
    {
        std::exception_ptr const p = std::current_exception();
        if ( false == static_cast<bool>(p) )
        {
            std::puts("exception_ptr is invalid - might be forced unwind");
            throw;
        }
    }

    return nullptr;
}

int main(void)
{
    ::pthread_t h;
    ::pthread_create(&h, nullptr, Thread_Entry_Point, nullptr);
    ::sleep(3u);
}
```

```

::pthread_cancel(h);
::pthread_join(h, nullptr);

```

In the case of a forced unwinding, the recommendation is that `exception_typeid()` shall return either `typeid(void)`, or the `type_info` of a special type which indicates forced unwinding, for example `typeid(abi::__forced_unwind)` -- even if `current_exception()` returns a null pointer. This is a scenario in which `exception_typeid()` might not be equivalent to `exception_typeid( current_exception() )`.

In a forced unwinding, `current_exception()` returns a null pointer. The recommendation is that `exception_object()` shall return `nullptr` in any circumstance where `current_exception()` returns a null pointer.

§ 6. Possible implementations

§ 6.1. Apple macOS, Linux, FreeBSD

```

#include <exception>    // exception_ptr, rethrow_exception
#include <memory>       // addressof
#include <typeinfo>     // type_info
#include <cxxabi.h>     // __cxa_current_exception_type

namespace std {
    type_info const &exception_typeid() noexcept
    {
        type_info const *const p = abi::__cxa_current_exception_type();
        if ( nullptr == p ) return typeid(void);
        return *p;
    }

    type_info const &exception_typeid(exception_ptr const &p) noexcept
    {
        if ( false == static_cast<bool>(p) ) return typeid(void);
        try { rethrow_exception(p); }
        catch(...) { return exception_typeid(); }
        return typeid(void); // not needed – but belt and braces
    }

    void *exception_object(exception_ptr const &p) noexcept

```

```

{
    if ( false == static_cast<bool>(p) ) return nullptr;
    return *(void**)addressof(p);
}

void *exception_object() noexcept
{
    return exception_object( current_exception() );
}
}

```

§ 6.2. Microsoft Windows

```

#include <cstdint>          // uintptr_t
#include <exception>        // exception_ptr, rethrow_exception
#include <memory>           // addressof
#include <typeinfo>         // type_info

#include <Windows.h>        // AddVectoredExceptionHandler
#include <ehdata.h>         // ThrowInfo, CatchableType, EXCEPTION_POINTER

thread_local std::type_info const *pti = &typeid(void);

namespace std {
    type_info const &exception_typeid() noexcept
    {
        return *pti;
    }

    type_info const &exception_typeid(exception_ptr const &p) noexcept
    {
        if ( false == static_cast<bool>(p) ) return typeid(void);
        try { rethrow_exception(p); }
        catch(...) { return exception_typeid(); }
        return typeid(void); // not needed – but belt and braces
    }

    void *exception_object(exception_ptr const &p) noexcept
    {
        if ( false == static_cast<bool>(p) ) return nullptr;
        return *(void**)addressof(p);
    }
}

```

```

    }

    void *exception_object() noexcept
    {
        return exception_object( current_exception() );
    }
}

namespace detail {

using std::type_info;

type_info const *VectoredExceptionHandler_Proper(EXCEPTION_POINTERS con
{
    using std::uintptr_t;

    if ( nullptr == arg ) return nullptr;

    EXCEPTION_RECORD const *const pexc = arg->ExceptionRecord;

    if ( nullptr == pexc ) return nullptr;

    switch ( pexc->ExceptionCode )
    {
    case 0x40010006 /*EXCEPTION_OUTPUT_DEBUG_STRING*/:
    case 0x406D1388 /*EXCEPTION_THREAD_NAME */:
        return nullptr;
    }

    if ( 0x19930520 /* magic number */ != pexc->ExceptionInformation[0]
        return nullptr;

    if ( pexc->NumberParameters < 3u ) return nullptr;

    uintptr_t const module = (pexc->NumberParameters >= 4u)
        ? pexc->ExceptionInformation[3u]
        : 0u;

    ThrowInfo const *const pthri =
        static_cast<ThrowInfo const*>(
            reinterpret_cast<void const*>(
                static_cast<uintptr_t>(pexc->ExceptionInformation[2u]))

```

```

    if ( nullptr == pthri ) return nullptr;

    if ( 0 == pthri->pCatchableTypeArray ) return nullptr;

    if ( 0u == (0xFFFFFFFFFu & pthri->pCatchableTypeArray) ) return null

    _CatchableTypeArray const *const pcarr =
        static_cast<_CatchableTypeArray const *>(
            reinterpret_cast<void const*>(
                static_cast<uintptr_t>(module + (0xFFFFFFFFFu & pthri->pCatcha

    if ( 0u == ( 0xFFFFFFFFFu &
        reinterpret_cast<uintptr_t>(pcarr->arrayOfCatchableTypes[0u])
        return nullptr;

    CatchableType const *const pct =
        static_cast<CatchableType const*>(
            reinterpret_cast<void const*>(
                static_cast<uintptr_t>(module +
                    (0xFFFFFFFFFu & reinterpret_cast<uintptr_t>(
                        pcarr->arrayOfCatchableTypes[0u]))));

    if ( 0u == (0xFFFFFFFFFu & pct->pType) ) return nullptr;

    return static_cast<type_info const *>(
        reinterpret_cast<void const*>(
            static_cast<uintptr_t>(module + (0xFFFFFFFFFu & pct->pTyp
}

long WINAPI VectoredExceptionHandler(EXCEPTION_POINTERS *const pointers
{
    type_info const *const retval = VectoredExceptionHandler_Proper(poi

    if ( nullptr == retval ) pti = &typeid(void);
        else pti = retval;

    return EXCEPTION_CONTINUE_SEARCH;
}

// The next line ensures that 'AddVectoredExceptionHandler' is called b
void *const dummy = ::AddVectoredExceptionHandler(1u, VectoredException

} // close namespace 'detail'

```

§ 7. Proposed wording

The proposed wording is relative to [\[N4950\]](#).

In section 17.9.7 [\[support.exception.propagation\]](#), append:

- 13** `type_info const &exception_typeid() noexcept;`
Returns: A reference to a 'type_info' (17.7.3) object for the currently handled exception, or 'typeid(void)' (7.6.1.8) if the currently handled exception is lacking 'type_info'.
- 14** `type_info const &exception_typeid(exception_ptr const &p) noexcept;`
Returns: A reference to a 'type_info' (17.7.3) object for the exception object referred by 'p' (17.9.7.1), or 'typeid(void)' (7.6.1.8) if 'p' is a null pointer, or 'typeid(void)' if the exception is lacking 'type_info'.
- 15** `void *exception_object() noexcept;`
Returns: The address of the exception object for the currently handled exception, or a null pointer if the exception is lacking an exception object.
- 16** `void *exception_object(exception_ptr const &p) noexcept;`
Returns: The address of the exception object for the exception referred by 'p' (17.9.7.1), or a null pointer if 'p' is a null pointer or a null pointer if the exception is lacking an exception object.

§ 8. Alternatives

§ 8.1. `std::exception_ptr::type()`

It has been suggested that `std::exception_ptr` should be given member functions, and that it should be defined in the standard as follows:

```
class exception_ptr {
public:
    type_info const &type() const noexcept;
    void *object() const noexcept;
};
```

This however would prevent the compiler from simply doing:

```
typedef shared_ptr<void> exception_ptr;
```

Furthermore, if `exception_ptr` is a non-polymorphic type, then all new member functions would have to be non-virtual in order to avoid an ABI break. Even if `exception_ptr` is already a polymorphic type, adding a virtual member function could result in an ABI break for derived classes such as:

```
class ExceptionHandle : public std::exception_ptr {
public:
    virtual void output(std::ostream&) const;
};
```

Adding a new virtual member function to the class `exception_ptr` will have an effect on the index of `output` in the vtable belonging to `ExceptionHandle` resulting in an ABI break.

If we were to add a new member function to the `exception_ptr` class instead of creating a standalone function `std::exception_typeid`, then we would have no way of getting the `type_info` in situations where `current_exception()` returns a null pointer (such as a forced unwinding). Therefore even if we add a member function to `exception_ptr`, we still need the standalone function `std::exception_typeid()`. For the sake of simplicity it makes more sense just to have the standalone function.

§ 9. Acknowledgements

For their feedback and contributions on the mailing list std-proposals@lists.isocpp.org:

Tom Honermann, Thiago Macieira, Jens Maurer, Jason McKesson, Arthur O'Dwyer, Peter Olsson, Jan Schultke, Lénárd Szolnoki, Matthew Taylor, Ville Voutilainen, Sebastian Wittmeier and [the late Edward Catmur \(1982-2024\)](#).

§ 10. Revisions

R0 => R1

- Revision number incremented to compensate for a corrupt upload of the original paper. (No other changes).

§ References

§ Normative References

[N4950]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 10 May 2023. URL: <https://wg21.link/n4950>

§ Informative References

[P0933R1]

Aaryaman Sagar. *Runtime type introspection with `std::exception_ptr`*. 7 February 2018. URL: <https://wg21.link/p0933r1>

[P1066R1]

Mathias Stearn. *How to catch an `exception_ptr` without even try-ing*. 6 October 2018. URL: <https://wg21.link/p1066r1>

[P2927R1]

Arthur O'Dwyer, Gor Nishanov. *Inspecting `exception_ptr`*. 14 February 2024. URL: <https://wg21.link/p2927r1>